

tf.compat.v1.gather Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/gather

Gather slices from params axis axis according to indices. (deprecated arguments)

Function Signatures

```
tf.compat.v1.gather( params, indices, validate_indices=None,  
name=None, axis=None, batch_dims=0 )  
(validate_indices)
```

Code Examples

```
tf.compat.v1.gather(  
    params, indices, validate_indices=None, name=None, axis=None,  
    batch_dims=0  
)
```

```
tf.compat.v1.gather(  
    params, indices, validate_indices=None, name=None, axis=None,  
    batch_dims=0  
)
```

```
params = tf.constant(['p0', 'p1', 'p2', 'p3', 'p4', 'p5'])  
params[3].numpy()  
b'p3'  
tf.gather(params, 3).numpy()  
b'p3'
```

```
params = tf.constant(['p0', 'p1', 'p2', 'p3', 'p4', 'p5'])
```

```
params[3].numpy()
```

```
tf.gather(params, 3).numpy()
```

Documentation

Gather slices from `params` `axis` `axis` according to `indices`. (deprecated arguments)

Used in the notebooks

- Linear Mixed-Effect Regression in {TF Probability, R, Stan}

Gather slices from params axis axis according to indices. indices must be an integer tensor of any dimension (often 1-D).

Tensor.getitem works for scalars, tf.newaxis, and python slices

tf.gather extends indexing to handle tensors of indices.

In the simplest case it's identical to scalar indexing:

The most common case is to pass a single axis tensor of indices (this can't be expressed as a python slice because the indices are not sequential):

The indices can have any shape. When the params has 1 axis, the output shape is equal to the input shape:

The params may also have any shape. gather can select slices across any axis depending on the axis argument (which defaults to 0). Below it is used to gather first rows, then columns from a matrix:

More generally: The output shape has the same shape as the input, with the indexed-axis replaced by the shape of the indices.

Here are some examples:

This is because each index takes a slice from params, and places it at the corresponding location in the output. For the above example

Batching:

The `batch_dims` argument lets you gather different items from each element of a batch.

Using `batch_dims=1` is equivalent to having an outer loop over the first axis of params and indices:

This is equivalent to:

Higher values of `batch_dims` are equivalent to multiple nested loops over the outer axes of params and indices. So the overall shape function is

This comes up naturally if you need to use the indices of an operation like `tf.argsort`, or `tf.math.top_k` where the last dimension of the indices indexes into the last dimension of input, at the corresponding location. In this case you can use `tf.gather(values, indices, batch_dims=-1)`.

See also:

- `tf.Tensor.getitem`: The direct tensor index operation (`t[i]`), handles scalars and python-slices `tensor[..., 7, 1:-1]`
 - `tf.scatter`: A collection of operations similar to `__setitem__`

(`t[i] = x`)

- `tf.gather_nd`: An operation similar to `tf.gather` but gathers across

multiple axis at once (it can gather elements of a matrix instead of rows or columns)

- `tf.boolean_mask`, `tf.where`: Binary indexing.

- `tf.slice` and `tf.strided_slice`: For lower level access to the

implementation of `__getitem__`'s python-slice handling (`t[1:-1:2]`)

Returns